Ivan N Guzman

Programacion Logica y Funcional

Calculo Lambda (λ)

Es un sistema formal de lógica matemática diseñada para expresar la computación basándose únicamente en la abstracción y aplicación de funciones. Alonzo Church lo creo en 1936 para investigar los fundamentos de las matemáticas (especificamente el Entscheidungsproblem). Es equivalente matemático de la máquina de Turing; es decir, cualquier algoritmo computable puede expresarse puramente con el cálculo lambda (Tesis de church-Turing).

Sintaxis Básica (Reglas de formación)

Todo el cálculo lambda se construye con un conjunto de reglas muy restrictivo llamado "terminos lambda". Solo hay 3 componentes:

- Variables: Letras que representan parámetros (ej. x, y, z)
- Abstracción (Creación de funciones): Se utilizan para definir funciones anónimas. Su estructura es $(\lambda x.M)$, donde x es el parámetro ligado y M es el cuerpo de la función.
- Aplicación (Llamada a funciones): Es el acto de evaluar una función con un argumento. Su estructura es $(M\ N)$, que significa "aplica la función M al argumento N".

Evaluando expresiones: Reducción Beta (β)

Es la operación principal del sistema. Consiste en "ejecutar" la función sustituyendo las instancias de la variable en el cuerpo de la función por el argumento que se la está pasando. Matemáticamente se expresa así:

$$(\lambda x.M)\,N \rightarrow M[x := N]$$

Importancia en la Programación Funcional

Aunque nació como un modelo teórico puro décadas antes de que existieran las computadoras modernas, el cál-

culo lambda es la base absoluta del paradigma funcional. Lenguajes históricos como Lisp y modernos como Haskell se construyeron sobre este modelo. De hecho, las funciones lambda que hoy programas en Python derivan directamente de este trabajo

Referencias

Barendregt, H. P. (1984). The Lambda Calculus: Its Syntax and Semantics. Studies in Logic and The Foundations of Mathematics (Vol. 103). North-Holland.

Church, A. (1936). An Unsolvable Problem of Elementary Number Theory. American Journal of Mathematics, 58(2), 345-363

Repositorio Institucional DGB-UNAM. Cálculo Lambda, el origen de la programación (Capítulo de análisis de lenguajes formales.

05 Oct 26

Programación Funcional

Basada en funciones matemáticas

X cambios de estado y cambios de valores

- Imperativo: Pasos detallados sobre como hacer algo
- Funcional: que hacer utilizando funciones que transforman datos

Inmutabilidad: No se puede modificar variables o estructura de datos.

Funciones puras: mismos datos de entrada devuelven los mismos resultados

Funciones como ciudadanos de primera clase: Poder tratar a funciones como cualquier otro dato.

Recursividad
Una forma que puede repetir algo

Imperativa: contadores, while, for

Funcional: El problema se divide en dos partes
- Caso base: Condición de salida.
- Caso recursivo

Programación Lógica y Funcional

Erlang

Origen

Fue creado en 1986 dentro de los laboratorios Ericsson. Nació de una necesidad muy específica: controlar conmutadores telefónicos y redes de telecomunicaciones donde una interrupción del servicio era inaceptable.

Plataforma y BEAM

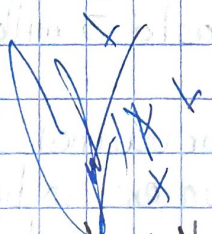
El código de Erlang se compila para ejecutarse sobre la máquina virtual BEAM. A diferencia de las máquinas de Java o Python, BEAM está diseñada de raíz para la concurrencia; esto significa que distribuye la carga de trabajo automáticamente entre todos los núcleos del procesador de forma muy eficiente.

Filosofía "Let it crash"

En lenguajes tradicionales, pasas mucho tiempo escribiendo código para prevenir y cachar errores. En Erlang, la filosofía es usar "árboles de supervisión". Si un proceso falla, no intentas rescatarlo; lo dejas morir. Un proceso supervisor detecta la caída, limpia la memoria y lo reinicia desde un estado seguro casi instantaneamente.

Procesos ligeros.

Erlang no utiliza los hilos (threads) tradicionales del sistema operativo. Implementa sus propios "procesos" internos que consumen apenas unos 300 bytes. Esto elimina los cuellos de botella de memoria RAM, permitiendo levantar literalmente millones de procesos en un solo servidor físico.



Elixir

Origen

Fue diseñado en 2012 por José Valim. Como desarrollador del ecosistema Ruby, Valim quería mantener la productividad y sintaxis limpia de Ruby, pero llevarla al entorno robusto y escalable de la máquina virtual de Erlang (BEAM).

Propósito

Elixir toma todo el poder industrial de Erlang y le añade características de lenguajes modernos. Destaca su uso de metaprogramación (macros), que permite a los programadores escribir código que genera más código, haciendo los proyectos mucho más fáciles de leer y mantener

Ecosistema web (Phoenix).

Elixir brilla en el desarrollo web gracias a su framework Phoenix. Aprovechando los procesos ligeros de BEAM, Phoenix es capaz de mantener millones de conexiones WebSockets abiertas simultáneas. Esto lo hace ideal para aplicaciones en tiempo real, tableros interactivos y plataformas de streaming.

Uso en la arquitectura de WhatsApp

WhatsApp es el caso de estudio definitivo sobre la capacidad de Erlang/BEAM para escalar con pocos recursos.

Eficiencia de equipo

En 2014, cuando WhatsApp manejaba miles de millones de mensajes diarios, su equipo de ingeniería de backend apenas rondaba los 35 programadores. Esto fue posible porque Erlang ya resolvía por defecto la complejidad de los sistemas distribuidos.

Conexiones simultáneas

Al inicio, WhatsApp modificó ejabberd (un servidor de chat de código abierto escrito en Erlang). Posteriormente, optimizaron tanto la máquina BEAM como el sistema operativo para lograr que un solo servidor soportara hasta 3 millones de conexiones TCP activas de manera estable.

Aislamiento (Un proceso por usuario)

La arquitectura asigna un proceso ligero exclusivo a cada usuario conectado. Si el envío de un archivo corrupto provoca un fallo, solo el proceso de este usuario específico colapsa y se reinicia en milisegundos. El resto de los millones de usuarios en ese servidor ni se enteran.

Actualizaciones "en caliente" (Hot Code Swapping)

WhatsApp mantiene un uptime (tiempo en línea) extremo del 99.999% porque Erlang permite inyectar o actualizar el código de la aplicación mientras el servidor sigue corriendo. No necesitan "bajar el sistema por mantenimiento" para aplicar parches de seguridad

Referencia

Armstrong, J. (2003). Making reliable distributed systems in the presence of software errors [Tesis doctoral, Royal Institute of Technology]. DiVA Portal. https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-3569

Reed, R. (2014, 27 de marzo). That's 'Billion' with a 'B': Scaling to the next level at WhatsApp [Presentación de conferencia]. Erlang Factory SF Bay Area 2014, San Francisco, CA, Estados Unidos.

Valim, J. (2024). Elixir Documentation: Introduction. Elixir lang. Recuperado el 5 de Octubre de 2026, de https://hexdocs.pm/elixir/introduction.html

Programacion Lógica y Funcional

Funciones de orden superior

En los paradigmas de programacion funcional y lógica, las funciones son tratadas como ciudadanos de primera clase (first-class citizens). Esto implica que pueden ser asignadas a variables, pasadas como argumentos y retornadas por otras funciones. Una Funcion de Orden Superior (FOS) es aquella que acepta una o más funciones como parámetros de entrada, o en su defecto, devuelve una función como resultado. El uso de FOS fomenta un enfoque declarativo, donde se describe de manera abstracta qué se quiere calcular, eliminando la necesidad de definir algoritmicamente cómo iterar sobre las estructuras de datos.

Las tres funciones de orden superior más fundamentales para el procesamiento de listas son MAP, Filter y Reduce.

1. Función Map (Proyección o Transformación)

La función Map aplica una función unaria (de un solo parámetro) de forma independiente a cada uno de los elementos de una colección o lista. Matemáticamente, si se dispone de una función $f: A \to B$ y una lista de tipo $[A]$, la operación map proyectará una nueva lista de tipo $[B]$.

- **Comportamiento clave:** Map conserva la longitud de la estructura original pero transforma sus valores. Debido al principio de inmutabilidad, no muta la lista original en memoria, sino que genera una nueva estructura con las proyecciones correspondientes.
- **Ejemplo en Haskell**

```
--Se aplica una función anónima para elevar cada elemento al
    cuadrado.
map (\x -> x^2) [1, 2, 3, 4]
-- Resultado: [1, 4, 9, 16]
```

- **Ejemplo equivalente en Python**

```
list(map(lambda x: x**2, [1, 2, 3, 4]))
```

2. Función Filter (Selección o Filtrado)

La función Filter se encarga de extraer elementos de una colección basándose en un criterio de selección estricto. Recibe como argumentos un predicado (una función que siempre retorna un valor lógico booleano) y una lista. Se evalúa cada elemento de forma individual; si el predicado retorna verdadero, el elemento se preserva en una lista; de lo contrario, se descarta.

- Comportamiento clave: La lista resultante será siempre de una longitud menor o igual a la original, conservando intacto el tipo de dato subyacente de los elementos seleccionados.
- Ejemplo en Racket (Scheme)

```
;; Uso del predicado interno even? para filtrar números pares.
(filter even? '(1 2 3 4 5 6))
;; Resultado: '(2 4 6)
```

- Ejemplo en Haskell

```
filter (>3) [1, 2, 3, 4, 5]
-- Resultado: [4, 5]
```

3. Función Reduce o Fold (Agregación)

Mientras que map y filter operan con una cardinalidad de 1 a 1, Reduce (conocido formalmente en lenguajes puros como Fold) condensa los elementos de una lista para producir un único valor consolidado o escalar. Requiere tres componentes: una función binaria, un valor inicial (acumulador) y la lista a procesar.

La reducción puede ejecutarse iterando de izquierda a derecha (foldl) o de derecha a izquierda (foldr). En cada paso, la función binaria procesa el estado actual del acumulador y el siguiente elemento de la lista para actualizar el estado del acumulador hasta consumir toda la estructura.

- Comportamiento clave: Transforma una secuencia de datos en un solo dato primitivo o en una nueva estructura compleja, absorbiendo toda la colección iterativamente.
- Ejemplo en Haskell (foldl)

```
-- Sumatoria de todos los elementos, iniciando con el acumulador en 0
foldl (+) 0 [1, 2, 3, 4]
-- Resultado: 10
```

- Ejemplo en Python

```
from functools import reduce
reduce (lambda acc, x: acc + x, [1, 2, 3, 4], 0)
```

Importancia en el paradigma Funcional.

El uso orquestado de estas tres funciones permite reemplazar estructuras de control imperativas tradicionales (ciclos for o while) y sus variables de control. Al componer map, filter y reduce, se construyen flujos de datos (pipelines) seguros que previenen la mutación de estados y los efectos secundarios indeseados (side-effects). Además, este modelo matemático es la base de arquitecturas distribuidas modernas orientadas al procesamiento masivo de datos.

Referencias

Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and interpretation of computer programs (2.ª ed.) MIT Press.

Hutton, G. (2016). Programming in Haskell (2.ª ed.) Cambridge University Press.

Sebesta, R. W. (2019). Concepts of programming languages (12.ª ed.) Pearson.

Aspecto	Prefija	Infija	Postfija
Definición.	El nombre de la función va antes de sus argumentos.	El nombre (operador) va entre sus dos argumentos	El nombre va después de su argumento.
Soporte en Hs	Es la forma por defecto de toda función con nombre alfanumérico.	Nativa para operadores (símbolos) y para funciones usando acentos graves.	No existe nativamente. Se simulan operadores o extensiones
	f x y	x + y o x `f` y	x & f (simulada)
Ejemplo	max 3 7 → 7	3+7 → 10 10 `div` 3 → 3	[1,2,3] & length → 3 (con DataFunction)
Precedencia y Asociatividad	Aplicación de función: la mayor precedencia (10), asocia a la izquierda (f x y = (f x) y).	Se controla con infixl, infixr o infix y un nivel de 0 a 9.	Depende del operador que se defina; si se simula, se declara su fixity.
Ventajas	Sintaxis uniforme, ideal para aplicación parcial y currying (map (add 1))	Más legible en operaciones matemáticas y lógicas.	Lectura de izquierda a derecha, estilo "pipeline".
Desventajas	Expresiones anidadas se vuelven difíciles de leer	Hay que conocer la precedencia de cada operador para no equivocarse.	No es estándar, requiere importar o definir el operador.
Exmplos Típicos	sum, map, filter, foldr, succ	+, −, *, &&, ++, ., $, :	& (Data.Function), operadores definidos como 1?, extensión PostfixOperators

Iván N. Guzmán

Programación Lógica y Funcional | 3 Nov 26

2.3 Intervalos

Un intervalo (range) es una notación que permite construir listas a partir de secuencias aritméticas, sin escribir cada elemento de forma manual. En Haskell, la expresión [1..10] genera la lista de los enteros del 1 al 10, mientras que [2,4..10] indica un paso definido por los dos primeros elementos y produce [2, 4, 6, 8, 10] (Lipovača, 2011). Según el Haskell 2010 Language Report, estas expresiones son azúcar sintáctica de las funciones enumFrom, enumFromThen, enumFromTo y enumFromThenTo, que pertenece a la clase de tipo Enum (Marlow, 2010). Por ello, los intervalos no se limitan a números enteros, sino que funcionan con cualquier tipo que se instancia de Enum, como los caracteres ['a'..'e'] produce "abcde".

Los intervalos también admiten pasos descendentes, como [10,8..0], y pueden ser infinitos si se omite el límite superior, como en [1..]. Esto es solo práctico gracias a la evaluación perezosa, pues la lista se construye conforme se consume; por ejemplo, take 5 [1..] devuelve [1, 2, 3, 4, 5] (Hutton, 2016). Con números decimales conviene tener cuidado, ya que el reporte del lenguaje indica que la secuencia puede incluir elementos ligeramente mayores al límite superior (Marlow, 2010).

2.4 Operadores

En Haskell, un operador es una función cuyo nombre está formado por símbolos y que se escribe por defecto en forma infija, por lo que 2 + 3 equivale (+) 2 3 (Marlow, 2010). Del mismo modo cualquier función de dos argumentos puede usarse como operador si se encierra entre acentos graves, como en 10 `div` 3 (Lipovača, 2011).

Cada operador tiene una precedencia (de 0 a 9) y una asociatividad (izquierda, derecha o ninguna), que se definen como declaraciones de fijeza (infixl, infixr, e infix). Si el programador no declara la fijeza de un operador propio, se

asume infixl 9 (Marlow, 2010). Por ejemplo, * tiene mayor precedencia (7) que + (6), por lo que 2 + 3 * 4 da 14.

También existen las secciones, que son la aplicación parcial de un operador infijo: (+1) suma uno a su argumento y (2*) lo multiplica por dos (Hutton, 2026). Finalmente, el operador . compone funciones, de modo que (f . g) x equivale a f (g x), y $ evita paréntesis por tener la precedencia más baja (Lipovača, 2011). Hughes (1989) sostiene que combinar funciones pequeñas con este tipo de operadores es la base de la modularidad en la programación funcional.

2.7 Evaluación perezosa.

La evaluación perezosa es una estrategia en la que una expresión no se evalúa hasta que su valor es necesario y, una vez evaluada, el resultado se comparte para no repetir el cálculo (Hutton, 2016). En Haskell esto es habitual, y Hudak et al. (2007) señalan que fue el principal motivo que llevó a mantener el lenguaje puro.

Sus ventajas principales son tres. Primero, evita cálculos innecesarios: fst (1, undefined) devuelve 1 sin error. Segundo, permite estructuras infinitas, como fibs = 0 : 1 : zipWith (+) fibs (tail fibs) de las que se consume solo lo necesario con take 10 fibs (Hutton, 2016). Tercero, mejora la modularidad, porque separa el generador de datos del consumidor que decide cuántos necesita (Hughes, 1989).

Como desventaja, el sistema debe guardar expresiones pendientes (thunks), lo que puede causar un alto consumo de memoria, conocido como fugas de espacio. Para controlarlo, Haskell ofrece la función seq y el operador $!, que fuerzan la evaluación de un argumento (Hutton, 2016).

Referencias

Hudak, P., Hughes, J., Peyton Jones, S., & Wadler, P. (2007). A

JEAN BOOK

history of Haskell: Being lazy with class. En Proceedings of the third ACM SIGPLAN conference on History of programming languages (pp. 12-1-12-55). ACM https://doi.org/10.1145/1238844.1238856

Hughes, J. (1989). Why functional programming matters. The Computer Journal, 32(2), 98-107. https://doi.org/10.1093/comjnl/32.2.98

Hutton, G. (2016). Programming in Haskell (2ª ed.) Cambridge University Press

Lipovača, M. (2011). Learn you a Haskell for great good! A beginners guide. No starch Press. https://learnyouahaskell.github.io/

Marlow, S. (Ed). (2010). Haskell 2010 language report. Haskel.org https://www.haskell.org/onlinereport/haskell2010/